User:

design a web dashboard that able to add modular windows on the fly and present different types of output, able to connect to different databases and api's create a document of technical design make all asyncronous dockerized able to switch different themes make a panels with refresh option add user management with roles able to manage different layout baeed on json definition define a DB to store it add loggin add versioning

---

Kimi:

I'll create a comprehensive technical design document for a modular web dashboard system. Let me first search for current best practices and relevant architectural patterns to ensure the design is modern and robust.Based on my research of modern architectural patterns and best practices, I'll now create a comprehensive technical design document for your modular dashboard system. This will be a production-ready, enterprise-grade architecture.

Modular Dashboard Platform (MDP)

Technical Design Document v1.0

---

1. Executive Summary

The Modular Dashboard Platform (MDP) is a scalable, containerized web application that enables users to create dynamic, personalized dashboards with modular windows (widgets) that can connect to various data sources (databases, APIs). The system features real-time asynchronous operations, multi-tenant user management with role-based access control (RBAC), theme switching, layout persistence via JSON definitions, comprehensive logging, and versioning capabilities.

Key Architectural Decisions: - **Frontend**: React 18+ with TypeScript, React-Grid-Layout for drag-drop-resize functionality - **Backend**: Node.js (Express/Fastify) or Python (FastAPI) with async/await patterns throughout - **Database**:

Generated by Kimi.ai

PostgreSQL (primary) + Redis (caching/sessions) + MongoDB (layout storage) -
Containerization: Docker with multi-stage builds for optimized images -
Orchestration: Docker Compose (dev) / Kubernetes (production)

---

2. System Architecture

2.1 High-Level Architecture Diagram

```
┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│                                                                                                                │
│                                                                                                                │
│                                                                                                                │
```

```
┌──────────────
```

```
Adapters   │      │
│  └──────┴──────┘ └──────┴──────┘ └──────┴──────┘
└──────┴──────┘      │
│        │              │              │              │
│
│        └──────────────┴──────────────┴──────────────┘
│
│                              │
│
│                       ┌──────┴──────┐
│
│                       │  Message Broker   │
│
│                       │  (Redis/RabbitMQ) │
│
│                       │  • Async Jobs     │
│
│                       │  • Event Bus      │
│
│                       │  • Caching        │
│
│                       └──────────────────┘
│
└──────────────────────────────────────────────────────────────
────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────────
────────────┐
│                        DATA LAYER
│
│  ┌──────────────┐ ┌──────────────┐ ┌──────────────┐
```

Generated by Kimi.ai

```
┌──────────────┐       │
│  │  PostgreSQL  │ │    Redis     │ │   MongoDB    │ │  MinIO/S3
│      │
│  │             │ │              │ │              │ │
│      │
│  │ • Users      │ │ • Sessions   │ │ • Layouts    │ │ • Assets
│      │
│  │ • Roles      │ │ • Cache      │ │ • Widgets    │ │ • Exports
│      │
│  │ • Audit Logs │ │ • Pub/Sub    │ │ • Versions   │ │ • Backups
│      │
│  │ • Connectors │ │ • Rate Limit │ │ • JSON Docs  │ │
│      │
│  └──────────────┘ └──────────────┘ └──────────────┘
└──────────────┘       │
└──────────────────────────────────────────────────────────────
────────────┘
```

Generated by Kimi.ai

2.2 Service Breakdown

Service	Responsibility	Tech Stack	Port
API Gateway	Routing, auth middleware, rate limiting	Nginx/Traefik	80/443
Auth Service	Authentication, authorization, user/role management	Node.js + Passport.js	3001
Layout Service	Dashboard layout CRUD, versioning, theme management	Node.js + MongoDB	3002
Widget Service	Widget lifecycle, rendering engines, refresh orchestration	Node.js + WebSockets	3003
Connector Service	Database/API connections, query execution, data transformation	Python + FastAPI + SQLAlchemy	3004
Logging Service	Centralized logging, log aggregation	Winston/Pino + ELK Stack	3005
Notification Service	Real-time updates, alerts	Socket.io + Redis	3006

---

3. Database Schema Design

3.1 PostgreSQL (Relational Data)

```
-- Core User Management & RBAC
CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email VARCHAR(255) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    first_name VARCHAR(100),
```

```sql
    last_name VARCHAR(100),
    avatar_url TEXT,
    is_active BOOLEAN DEFAULT true,
    is_superuser BOOLEAN DEFAULT false,
    last_login TIMESTAMP,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    deleted_at TIMESTAMP  -- Soft delete
);

CREATE TABLE roles (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name VARCHAR(50) UNIQUE NOT NULL,  -- admin, editor, viewer,
custom
    description TEXT,
    permissions JSONB NOT NULL,  -- Flexible permission structure
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE user_roles (
    user_id UUID REFERENCES users(id) ON DELETE CASCADE,
    role_id UUID REFERENCES roles(id) ON DELETE CASCADE,
    assigned_by UUID REFERENCES users(id),
    assigned_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    expires_at TIMESTAMP,  -- Optional time-bound roles
    PRIMARY KEY (user_id, role_id)
);

-- Teams/Organizations (Multi-tenancy support)
CREATE TABLE organizations (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

Generated by Kimi.ai

```sql
    name VARCHAR(255) NOT NULL,
    slug VARCHAR(100) UNIQUE NOT NULL,
    settings JSONB DEFAULT '{}',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE organization_members (
    organization_id UUID REFERENCES organizations(id) ON DELETE
CASCADE,
    user_id UUID REFERENCES users(id) ON DELETE CASCADE,
    role VARCHAR(50) NOT NULL,  -- owner, admin, member
    joined_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (organization_id, user_id)
);

-- Data Connectors
CREATE TABLE connectors (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    organization_id UUID REFERENCES organizations(id) ON DELETE
CASCADE,
    name VARCHAR(255) NOT NULL,
    type VARCHAR(50) NOT NULL,  -- postgresql, mysql, mongodb,
rest_api, graphql, etc.
    config JSONB NOT NULL,  -- Encrypted connection parameters
    credentials_encrypted TEXT,  -- AES-256 encrypted credentials
    is_active BOOLEAN DEFAULT true,
    health_status VARCHAR(20) DEFAULT 'unknown',
    last_health_check TIMESTAMP,
    created_by UUID REFERENCES users(id),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

Generated by Kimi.ai

```sql
);

-- Audit Logging
CREATE TABLE audit_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES users(id),
    organization_id UUID REFERENCES organizations(id),
    action VARCHAR(50) NOT NULL,  -- create, update, delete,
login, export
    entity_type VARCHAR(50) NOT NULL,  -- layout, widget,
connector, user
    entity_id UUID,
    old_values JSONB,
    new_values JSONB,
    ip_address INET,
    user_agent TEXT,
    timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Version Control for Layouts
CREATE TABLE layout_versions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    layout_id UUID NOT NULL,  -- References MongoDB document ID
    version_number INTEGER NOT NULL,
    created_by UUID REFERENCES users(id),
    change_summary TEXT,
    layout_snapshot JSONB NOT NULL,  -- Complete layout JSON
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    UNIQUE(layout_id, version_number)
);
```

Generated by Kimi.ai

```sql
-- Indexes for performance
CREATE INDEX idx_users_email ON users(email);
CREATE INDEX idx_audit_logs_user ON audit_logs(user_id, timestamp
DESC);
CREATE INDEX idx_audit_logs_org ON audit_logs(organization_id,
timestamp DESC);
CREATE INDEX idx_connectors_org ON connectors(organization_id);
```

3.2 MongoDB (Document Store - Layouts & Widgets)

```jsonc
// Dashboard Layouts Collection
{
  "_id": ObjectId("..."),
  "organization_id": "uuid-string",
  "created_by": "uuid-string",
  "title": "Sales Dashboard",
  "description": "Real-time sales metrics",
  "slug": "sales-dashboard",
  "is_default": false,
  "is_public": false,
  "shared_with": ["user-uuid-1", "user-uuid-2"],

  // Layout Definition (React-Grid-Layout format)
  "layout_config": {
    "breakpoints": {
      "lg": 1200, "md": 996, "sm": 768, "xs": 480, "xxs": 0
    },
    "cols": { "lg": 12, "md": 10, "sm": 6, "xs": 4, "xxs": 2 },
    "rowHeight": 100,
    "margin": [10, 10],
    "containerPadding": [10, 10],
    "compactType": "vertical"
  },
```

Generated by Kimi.ai

```
  // Widgets Array
  "widgets": [
    {
      "id": "widget-uuid-1",
      "type": "chart",  // chart, table, metric, text, iframe,
custom
      "title": "Revenue Trend",
      "x": 0, "y": 0, "w": 6, "h": 4,
      "minW": 2, "minH": 2, "maxW": 12, "maxH": 8,
      "static": false,
      "isDraggable": true,
      "isResizable": true,

      // Widget Configuration
      "config": {
        "dataSource": {
          "connector_id": "uuid",
          "query": "SELECT * FROM sales WHERE date > $1",
          "parameters": ["30d"],
          "refreshInterval": 300000,  // 5 minutes in ms
          "cacheDuration": 60000
        },
        "visualization": {
          "chartType": "line",
          "colors": ["#1890ff", "#52c41a"],
          "axis": { "x": "date", "y": "revenue" }
        },
        "interactions": {
          "clickAction": "drillDown",
          "drillDownTarget": "widget-uuid-2"
```

Generated by Kimi.ai

```
        }
      },

      // Theme-specific overrides
      "themeOverrides": {
        "dark": { "backgroundColor": "#1f1f1f" },
        "light": { "backgroundColor": "#ffffff" }
      }
    }
  ],

  // Theme Configuration
  "theme": {
    "id": "default-dark",
    "primaryColor": "#1890ff",
    "backgroundColor": "#141414",
    "textColor": "#ffffff",
    "fontFamily": "Inter, sans-serif"
  },

  "tags": ["sales", "executive", "kpi"],
  "version": 5,
  "created_at": ISODate("2024-01-01"),
  "updated_at": ISODate("2024-03-13"),
  "deleted_at": null
}

// Widget Templates Collection (Reusable widgets)
{
  "_id": ObjectId("..."),
  "name": "Revenue KPI Card",
```

Generated by Kimi.ai

```
  "category": "financial",
  "type": "metric",
  "description": "Displays key revenue metrics with trend
indicator",
  "defaultConfig": { ... },
  "icon": "dollar-sign",
  "isOfficial": true,
  "created_by": "system",
  "usage_count": 1523
}
```

3.3 Redis (Caching & Real-time)

```
Key Patterns:
- session:{user_id} -> User session data (TTL: 24h)
- cache:query:{hash} -> Cached query results (TTL: 60s-1h based
on config)
- cache:layout:{layout_id} -> Serialized layout (TTL: 5min)
- ws:connections:{user_id} -> Active WebSocket connections
- rate_limit:{ip}:{endpoint} -> Rate limiting counters
- pubsub:layout_updates:{layout_id} -> Real-time layout change
events
- lock:layout_edit:{layout_id} -> Distributed locks for
concurrent editing
```

Generated by Kimi.ai

4. Frontend Architecture

4.1 Technology Stack

Layer	Technology	Purpose
Framework	React 18 + TypeScript	UI components with strict typing
State Management	Zustand + TanStack Query	Server state caching, client state
Layout Engine	React-Grid-Layout	Drag-drop-resize grid system
Styling	Tailwind CSS + CSS Variables	Theme switching, utility-first
Charts	Apache ECharts / Recharts	Data visualization
Tables	TanStack Table	Advanced data grids
WebSockets	Socket.io-client	Real-time updates
Build Tool	Vite	Fast development, optimized builds

4.2 Component Architecture

```
src/
├── components/
│   ├── Dashboard/
│   │   ├── DashboardContainer.tsx      # Main layout wrapper
│   │   ├── GridLayout.tsx            # React-Grid-Layout
integration
│   │   ├── WidgetRenderer.tsx        # Dynamic widget loader
│   │   └── Toolbar/
│   │       ├── AddWidgetButton.tsx
│   │       ├── ThemeSwitcher.tsx
│   │       ├── LayoutSelector.tsx
│   │       └── RefreshControls.tsx
│   │
```

Generated by Kimi.ai

```
│   ├── Widgets/
│   │   ├── BaseWidget.tsx            # Abstract base class
│   │   ├── ChartWidget/
│   │   ├── TableWidget/
│   │   ├── MetricWidget/
│   │   ├── TextWidget/
│   │   └── IFrameWidget/
│   │
│   ├── Connectors/
│   │   ├── ConnectionManager.tsx
│   │   ├── QueryBuilder.tsx
│   │   └── ConnectionTester.tsx
│   │
│   └── Common/
│       ├── ThemeProvider.tsx         # Theme context & switching
│       ├── ErrorBoundary.tsx
│       ├── LoadingStates.tsx
│       └── PermissionGate.tsx        # RBAC component
│
├── hooks/
│   ├── useLayout.ts                  # Layout CRUD operations
│   ├── useWidgetData.ts              # Data fetching with
refresh
│   ├── useTheme.ts                   # Theme management
│   ├── usePermissions.ts             # RBAC checks
│   └── useWebSocket.ts               # Real-time connection
│
├── services/
│   ├── api/                          # Axios instances per
service
│   ├── websocket/
```

Generated by Kimi.ai

```
│   └── connectors/                    # Database/API adapters
│
├── stores/
│   ├── layoutStore.ts                 # Zustand store
│   ├── authStore.ts
│   └── themeStore.ts
│
├── types/
│   ├── layout.ts
│   ├── widget.ts
│   ├── connector.ts
│   └── user.ts
│
└── utils/
    ├── asyncUtils.ts                  # Promise helpers, debounce
    ├── encryption.ts                  # Client-side encryption
    └── validators.ts
```

4.3 Theme System Implementation

```typescript
// Theme configuration with CSS variables
export interface Theme {
  id: string;
  name: string;
  colors: {
    primary: string;
    secondary: string;
    background: string;
    surface: string;
    text: string;
    textMuted: string;
    border: string;
    success: string;
```

Generated by Kimi.ai

```
    warning: string;
    error: string;
    info: string;
  };
  typography: {
    fontFamily: string;
    fontSize: {
      xs: string;
      sm: string;
      base: string;
      lg: string;
      xl: string;
      '2xl': string;
    };
  };
  spacing: {
    unit: number;
    scale: number[];
  };
  shadows: {
    sm: string;
    md: string;
    lg: string;
  };
  borderRadius: {
    sm: string;
    md: string;
    lg: string;
  };
}
```

Generated by Kimi.ai

```
// CSS Variables injection
export const applyTheme = (theme: Theme) => {
  const root = document.documentElement;
  Object.entries(theme.colors).forEach(([key, value]) => {
    root.style.setProperty(`--color-${key}`, value);
  });
  root.style.setProperty('--font-family',
theme.typography.fontFamily);
};
```

---

5. Backend Architecture

5.1 Async Implementation Patterns

All services use **strict async/await patterns** with no blocking operations:

```
// Node.js Example - Layout Service
import express from 'express';
import { Pool } from 'pg';
import { MongoClient } from 'mongodb';
import Redis from 'ioredis';
import { EventEmitter } from 'events';

// Database connection pools
const pgPool = new Pool({
  connectionString: process.env.DATABASE_URL,
  max: 20,  // Connection pooling
  idleTimeoutMillis: 30000
});

const mongoClient = new MongoClient(process.env.MONGODB_URL);
const redis = new Redis(process.env.REDIS_URL);

// Async middleware wrapper
```

```
const asyncHandler = (fn: Function) => (req: Request, res:
Response, next: NextFunction) => {
  Promise.resolve(fn(req, res, next)).catch(next);
};

// Layout Controller with full async operations
export class LayoutController {
  // Get layout with async caching
  async getLayout(req: Request, res: Response) {
    const { layoutId } = req.params;
    const userId = req.user.id;

    // Try cache first (async)
    const cached = await redis.get(`cache:layout:${layoutId}`);
    if (cached) {
      return res.json(JSON.parse(cached));
    }

    // Fetch from MongoDB (async)
    const layout = await
mongoClient.db().collection('layouts').findOne({
      _id: new ObjectId(layoutId),
      $or: [
        { created_by: userId },
        { shared_with: userId },
        { is_public: true }
      ],
      deleted_at: null
    });

    if (!layout) {
```

Generated by Kimi.ai

```
      throw new NotFoundError('Layout not found');
    }

    // Check permissions (async)
    const hasPermission = await
this.checkLayoutPermission(userId, layout, 'read');
    if (!hasPermission) {
      throw new ForbiddenError('Access denied');
    }

    // Populate cache (fire and forget, but handle errors)
    redis.setex(`cache:layout:${layoutId}`, 300,
JSON.stringify(layout))
      .catch(err => logger.error('Cache set failed', err));

    res.json(layout);
  }

  // Update layout with versioning (async transaction)
  async updateLayout(req: Request, res: Response) {
    const { layoutId } = req.params;
    const updates = req.body;
    const userId = req.user.id;

    // Distributed lock to prevent concurrent edits
    const lockKey = `lock:layout_edit:${layoutId}`;
    const lock = await redis.set(lockKey, userId, 'EX', 30,
'NX');

    if (!lock) {
      throw new ConflictError('Layout is being edited by another
```

Generated by Kimi.ai

```
user');
    }

    try {
      // Get current version
      const current = await
mongoClient.db().collection('layouts').findOne(
        { _id: new ObjectId(layoutId) },
        { projection: { version: 1, widgets: 1 } }
      );

      // Create version snapshot in PostgreSQL (async)
      await pgPool.query(
        `INSERT INTO layout_versions (layout_id, version_number,
created_by, change_summary, layout_snapshot)
         VALUES ($1, $2, $3, $4, $5)`,
        [layoutId, current.version + 1, userId,
updates.changeSummary || 'Update', current]
      );

      // Update in MongoDB (async)
      const result = await
mongoClient.db().collection('layouts').findOneAndUpdate(
        { _id: new ObjectId(layoutId) },
        {
          $set: {
            ...updates,
            version: current.version + 1,
            updated_at: new Date()
          }
        },
```

Generated by Kimi.ai

```
        { returnDocument: 'after' }
      );

      // Invalidate cache (async)
      await redis.del(`cache:layout:${layoutId}`);

      // Broadcast update to connected clients (async)
      await redis.publish(`pubsub:layout_updates:${layoutId}`,
JSON.stringify({
        type: 'layout_updated',
        layoutId,
        updatedBy: userId,
        timestamp: new Date()
      }));

      res.json(result.value);
    } finally {
      await redis.del(lockKey);  // Release lock
    }
  }

  // Async permission check
  private async checkLayoutPermission(userId: string, layout:
any, action: string): Promise<boolean> {
    if (layout.created_by === userId) return true;
    if (layout.is_public && action === 'read') return true;

    // Check user roles (async DB query)
    const { rows } = await pgPool.query(
      `SELECT r.permissions FROM users u
       JOIN user_roles ur ON u.id = ur.user_id
```

Generated by Kimi.ai

```
      JOIN roles r ON ur.role_id = r.id
      WHERE u.id = $1`,
     [userId]
    );

    return rows.some(row => row.permissions.includes(`layout:$
{action}`));
  }
}
```

5.2 Connector Service (Python FastAPI)

```
# connector_service/main.py
from fastapi import FastAPI, Depends, HTTPException,
BackgroundTasks
from sqlalchemy.ext.asyncio import AsyncSession,
create_async_engine
from sqlalchemy.orm import sessionmaker
import aioredis
import asyncpg
from typing import Dict, Any, List
import asyncio
from dataclasses import dataclass
import json
import aiohttp

app = FastAPI(title="Connector Service")

# Async database engines
async_engine = create_async_engine(
    "postgresql+asyncpg://user:pass@postgres/connectors",
    pool_size=20,
    max_overflow=0,
```

Generated by Kimi.ai

```python
    echo=False
)

AsyncSessionLocal = sessionmaker(
    async_engine, class_=AsyncSession, expire_on_commit=False
)

redis = aioredis.from_url("redis://redis:6379")

@dataclass
class ConnectionConfig:
    type: str  # postgresql, mysql, mongodb, rest, graphql
    host: str
    port: int
    database: str
    username: str
    password: str  # Encrypted, decrypted here
    options: Dict[str, Any]

class DatabaseConnector:
    def __init__(self, config: ConnectionConfig):
        self.config = config
        self.pool = None

    async def connect(self):
        if self.config.type == "postgresql":
            self.pool = await asyncpg.create_pool(
                host=self.config.host,
                port=self.config.port,
                database=self.config.database,
                user=self.config.username,
```

Generated by Kimi.ai

```
                password=self.config.password,
                min_size=5,
                max_size=20
            )
        # ... other database types

    async def execute_query(self, query: str, params: List[Any])
-> List[Dict]:
        if not self.pool:
            await self.connect()

        async with self.pool.acquire() as conn:
            if self.config.type == "postgresql":
                rows = await conn.fetch(query, *params)
                return [dict(row) for row in rows]
            # ... other implementations

    async def health_check(self) -> bool:
        try:
            async with self.pool.acquire() as conn:
                await conn.execute("SELECT 1")
            return True
        except:
            return False

    async def close(self):
        if self.pool:
            await self.pool.close()

class APIConnector:
    def __init__(self, config: ConnectionConfig):
```

Generated by Kimi.ai

```
        self.config = config
        self.session = None

    async def connect(self):
        timeout = aiohttp.ClientTimeout(total=30)
        self.session = aiohttp.ClientSession(timeout=timeout)

    async def execute_request(self, endpoint: str, method: str =
"GET",
                              payload: Dict = None) -> Dict:
        if not self.session:
            await self.connect()

        url = f"{self.config.host}:{self.config.port}{endpoint}"
        headers = {"Authorization": f"Bearer
{self.config.password}"}

        async with self.session.request(method, url,
json=payload,
                                        headers=headers) as
response:
            return await response.json()

    async def close(self):
        if self.session:
            await self.session.close()

# Connection pool manager
class ConnectionManager:
    def __init__(self):
        self.connections: Dict[str, Any] = {}
```

Generated by Kimi.ai

```
        self.lock = asyncio.Lock()

    async def get_connection(self, connector_id: str) -> Any:
        async with self.lock:
            if connector_id not in self.connections:
                # Load config from DB (async)
                config = await self.load_config(connector_id)
                connector = self.create_connector(config)
                await connector.connect()
                self.connections[connector_id] = connector
            return self.connections[connector_id]

    async def load_config(self, connector_id: str) ->
ConnectionConfig:
        # Async DB query to get decrypted config
        async with AsyncSessionLocal() as session:
            result = await session.execute(
                "SELECT config, credentials_decrypted FROM
connectors WHERE id = :id",
                {"id": connector_id}
            )
            row = result.fetchone()
            return ConnectionConfig(**row.config,
password=row.credentials_decrypted)

    def create_connector(self, config: ConnectionConfig):
        if config.type in ["postgresql", "mysql", "mongodb"]:
            return DatabaseConnector(config)
        elif config.type in ["rest", "graphql"]:
            return APIConnector(config)
        raise ValueError(f"Unknown connector type:
```

Generated by Kimi.ai

```
{config.type}")

manager = ConnectionManager()

@app.post("/query/{connector_id}")
async def execute_query(
    connector_id: str,
    query_request: QueryRequest,
    background_tasks: BackgroundTasks,
    user: User = Depends(get_current_user)
):
    """
    Execute query asynchronously with caching and refresh support
    """
    cache_key = f"cache:query:{hash(query_request.query +
str(query_request.params))}"

    # Check cache
    cached = await redis.get(cache_key)
    if cached and not query_request.skip_cache:
        return json.loads(cached)

    # Get connection and execute (async)
    connector = await manager.get_connection(connector_id)

    try:
        if connector.config.type in ["rest", "graphql"]:
            result = await connector.execute_request(
                query_request.query,
                query_request.method,
                query_request.payload
```

Generated by Kimi.ai

```
            )
        else:
            result = await connector.execute_query(
                query_request.query,
                query_request.params
            )

        # Cache result asynchronously
        if query_request.cache_duration > 0:
            await redis.setex(
                cache_key,
                query_request.cache_duration,
                json.dumps(result)
            )

        # Log query execution in background
        background_tasks.add_task(log_query_execution, user.id,
connector_id,
                                  query_request.query,
len(str(result)))

        return {
            "data": result,
            "cached": False,
            "execution_time_ms": 0,  # Calculate actual time
            "connector_type": connector.config.type
        }

    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
```

Generated by Kimi.ai

```python
@app.post("/refresh/{widget_id}")
async def refresh_widget(
    widget_id: str,
    background_tasks: BackgroundTasks,
    user: User = Depends(get_current_user)
):
    """
    Force refresh widget data, bypassing cache
    """
    # Get widget config (async)
    widget = await get_widget_config(widget_id)

    # Execute query (async)
    result = await execute_query(
        widget.data_source.connector_id,
        QueryRequest(
            query=widget.data_source.query,
            params=widget.data_source.parameters,
            skip_cache=True,
            cache_duration=widget.data_source.cache_duration
        ),
        background_tasks,
        user
    )

    # Broadcast to WebSocket subscribers (async)
    await redis.publish(f"widget:refresh:{widget_id}",
json.dumps(result))

    return result
```

Generated by Kimi.ai

```python
@app.get("/health/{connector_id}")
async def health_check(connector_id: str):
    connector = await manager.get_connection(connector_id)
    is_healthy = await connector.health_check()

    # Update health status in DB (async)
    await update_health_status(connector_id, is_healthy)

    return {"healthy": is_healthy, "connector_id": connector_id}
```

---

6. Docker Architecture

6.1 Multi-Stage Dockerfile (Frontend)

```dockerfile
# Dockerfile.frontend
# Stage 1: Dependencies
FROM node:20-alpine AS deps
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production && npm cache clean --force

# Stage 2: Builder
FROM node:20-alpine AS builder
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY . .
RUN npm run build  # Vite build

# Stage 3: Production
FROM nginx:alpine AS runner
# Copy built assets
COPY --from=builder /app/dist /usr/share/nginx/html
# Copy nginx config
```

```dockerfile
COPY nginx.conf /etc/nginx/conf.d/default.conf
# Security: Run as non-root
RUN addgroup -g 1001 -S nodejs && \
    adduser -S nextjs -u 1001 && \
    chown -R nextjs:nodejs /usr/share/nginx/html
USER nextjs
EXPOSE 80
HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --
retries=3 \
    CMD curl -f http://localhost/health || exit 1
```

6.2 Multi-Stage Dockerfile (Backend Node.js)

```dockerfile
# Dockerfile.backend
# Stage 1: Dependencies
FROM node:20-alpine AS deps
WORKDIR /app
COPY package*.json ./
RUN npm ci && npm cache clean --force

# Stage 2: Builder (TypeScript compilation)
FROM node:20-alpine AS builder
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY . .
RUN npm run build  # tsc compilation

# Stage 3: Production
FROM node:20-alpine AS runner
WORKDIR /app
ENV NODE_ENV=production
ENV PORT=3000
```

Generated by Kimi.ai

```
# Install only production dependencies
COPY package*.json ./
RUN npm ci --only=production && npm cache clean --force

# Copy compiled code
COPY --from=builder /app/dist ./dist

# Security hardening
RUN addgroup -g 1001 -S nodejs && \
    adduser -S nodejs -u 1001 && \
    chown -R nodejs:nodejs /app
USER nodejs

EXPOSE 3000
HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --
retries=3 \
    CMD node -e "require('http').get('http://localhost:3000/
health', (r) => r.statusCode === 200 ? process.exit(0) :
process.exit(1))"

CMD ["node", "dist/server.js"]
```

6.3 Docker Compose (Development)

```
# docker-compose.yml
version: '3.8'

services:
  # Frontend
  frontend:
    build:
      context: ./frontend
      dockerfile: Dockerfile
```

Generated by Kimi.ai

```yaml
    target: runner
  ports:
    - "80:80"
  environment:
    - REACT_APP_API_URL=http://localhost:8080
    - REACT_APP_WS_URL=ws://localhost:8080
  depends_on:
    - api-gateway
  networks:
    - dashboard-network

# API Gateway (Nginx)
api-gateway:
  image: nginx:alpine
  ports:
    - "8080:80"
  volumes:
    - ./nginx/nginx.conf:/etc/nginx/conf.d/default.conf:ro
  depends_on:
    - auth-service
    - layout-service
    - widget-service
    - connector-service
  networks:
    - dashboard-network

# Microservices
auth-service:
  build:
    context: ./backend/auth
    dockerfile: Dockerfile
```

Generated by Kimi.ai

```yaml
    environment:
      - NODE_ENV=development
      - DATABASE_URL=postgresql://
postgres:password@postgres:5432/dashboard
      - REDIS_URL=redis://redis:6379
      - JWT_SECRET=${JWT_SECRET}
      - JWT_EXPIRES_IN=24h
    volumes:
      - ./backend/auth:/app
      - /app/node_modules
    depends_on:
      postgres:
        condition: service_healthy
      redis:
        condition: service_healthy
    networks:
      - dashboard-network

  layout-service:
    build:
      context: ./backend/layout
      dockerfile: Dockerfile
    environment:
      - NODE_ENV=development
      - DATABASE_URL=postgresql://
postgres:password@postgres:5432/dashboard
      - MONGODB_URL=mongodb://mongo:27017/dashboard
      - REDIS_URL=redis://redis:6379
    volumes:
      - ./backend/layout:/app
      - /app/node_modules
```

Generated by Kimi.ai

```yaml
    depends_on:
      postgres:
        condition: service_healthy
      mongo:
        condition: service_healthy
      redis:
        condition: service_healthy
    networks:
      - dashboard-network

  widget-service:
    build:
      context: ./backend/widget
      dockerfile: Dockerfile
    environment:
      - NODE_ENV=development
      - REDIS_URL=redis://redis:6379
      - CONNECTOR_SERVICE_URL=http://connector-service:3004
    ports:
      - "3003:3003"  # WebSocket port
    depends_on:
      redis:
        condition: service_healthy
    networks:
      - dashboard-network

  connector-service:
    build:
      context: ./backend/connector
      dockerfile: Dockerfile
    environment:
```

Generated by Kimi.ai

```yaml
      - DATABASE_URL=postgresql://
postgres:password@postgres:5432/dashboard
      - REDIS_URL=redis://redis:6379
    volumes:
      - ./backend/connector:/app
    depends_on:
      postgres:
        condition: service_healthy
      redis:
        condition: service_healthy
    networks:
      - dashboard-network

  # Databases
  postgres:
    image: postgres:16-alpine
    environment:
      - POSTGRES_USER=postgres
      - POSTGRES_PASSWORD=password
      - POSTGRES_DB=dashboard
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./init-scripts:/docker-entrypoint-initdb.d:ro
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 10s
      timeout: 5s
      retries: 5
    networks:
      - dashboard-network
```

Generated by Kimi.ai

```yaml
  mongo:
    image: mongo:7-jammy
    volumes:
      - mongo_data:/data/db
    healthcheck:
      test: echo 'db.runCommand("ping").ok' | mongosh
localhost:27017/test --quiet
      interval: 10s
      timeout: 10s
      retries: 5
    networks:
      - dashboard-network

  redis:
    image: redis:7-alpine
    volumes:
      - redis_data:/data
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 10s
      timeout: 3s
      retries: 5
    networks:
      - dashboard-network

  # Logging & Monitoring
  elasticsearch:
    image: docker.elastic.co/elasticsearch/elasticsearch:8.11.0
    environment:
      - discovery.type=single-node
      - xpack.security.enabled=false
```

Generated by Kimi.ai

```yaml
      - "ES_JAVA_OPTS=-Xms512m -Xmx512m"
    volumes:
      - es_data:/usr/share/elasticsearch/data
    networks:
      - dashboard-network

  logstash:
    image: docker.elastic.co/logstash/logstash:8.11.0
    volumes:
      - ./logging/logstash.conf:/usr/share/logstash/pipeline/
logstash.conf:ro
    depends_on:
      - elasticsearch
    networks:
      - dashboard-network

  kibana:
    image: docker.elastic.co/kibana/kibana:8.11.0
    ports:
      - "5601:5601"
    depends_on:
      - elasticsearch
    networks:
      - dashboard-network

volumes:
  postgres_data:
  mongo_data:
  redis_data:
  es_data:
```

Generated by Kimi.ai

```yaml
networks:
  dashboard-network:
    driver: bridge
```

---

7. User Management & RBAC

7.1 Permission Structure

```typescript
// Permission definitions
export enum Permission {
  // Layout permissions
  LAYOUT_CREATE = 'layout:create',
  LAYOUT_READ = 'layout:read',
  LAYOUT_UPDATE = 'layout:update',
  LAYOUT_DELETE = 'layout:delete',
  LAYOUT_SHARE = 'layout:share',
  LAYOUT_PUBLISH = 'layout:publish',

  // Widget permissions
  WIDGET_CREATE = 'widget:create',
  WIDGET_READ = 'widget:read',
  WIDGET_UPDATE = 'widget:update',
  WIDGET_DELETE = 'widget:delete',
  WIDGET_REFRESH = 'widget:refresh',

  // Connector permissions
  CONNECTOR_CREATE = 'connector:create',
  CONNECTOR_READ = 'connector:read',
  CONNECTOR_UPDATE = 'connector:update',
  CONNECTOR_DELETE = 'connector:delete',
  CONNECTOR_TEST = 'connector:test',

  // User management
```

Generated by Kimi.ai

```
  USER_CREATE = 'user:create',
  USER_READ = 'user:read',
  USER_UPDATE = 'user:update',
  USER_DELETE = 'user:delete',
  USER_MANAGE_ROLES = 'user:manage_roles',

  // System
  SYSTEM_SETTINGS = 'system:settings',
  SYSTEM_LOGS = 'system:logs',
  SYSTEM_BACKUP = 'system:backup'
}

// Predefined roles
export const ROLES = {
  SUPER_ADMIN: {
    name: 'super_admin',
    permissions: Object.values(Permission),
    description: 'Full system access'
  },
  ADMIN: {
    name: 'admin',
    permissions: [
      Permission.LAYOUT_CREATE, Permission.LAYOUT_READ,
Permission.LAYOUT_UPDATE,
      Permission.LAYOUT_DELETE, Permission.LAYOUT_SHARE,
      Permission.WIDGET_CREATE, Permission.WIDGET_READ,
Permission.WIDGET_UPDATE,
      Permission.WIDGET_DELETE, Permission.WIDGET_REFRESH,
      Permission.CONNECTOR_CREATE, Permission.CONNECTOR_READ,
Permission.CONNECTOR_UPDATE,
      Permission.CONNECTOR_DELETE, Permission.CONNECTOR_TEST,
```

Generated by Kimi.ai

```
      Permission.USER_CREATE, Permission.USER_READ,
Permission.USER_UPDATE,
      Permission.SYSTEM_SETTINGS, Permission.SYSTEM_LOGS
    ]
  },
  EDITOR: {
    name: 'editor',
    permissions: [
      Permission.LAYOUT_CREATE, Permission.LAYOUT_READ,
Permission.LAYOUT_UPDATE,
      Permission.LAYOUT_SHARE,
      Permission.WIDGET_CREATE, Permission.WIDGET_READ,
Permission.WIDGET_UPDATE,
      Permission.WIDGET_DELETE, Permission.WIDGET_REFRESH,
      Permission.CONNECTOR_READ, Permission.CONNECTOR_TEST
    ]
  },
  VIEWER: {
    name: 'viewer',
    permissions: [
      Permission.LAYOUT_READ,
      Permission.WIDGET_READ, Permission.WIDGET_REFRESH,
      Permission.CONNECTOR_READ
    ]
  }
};
```

7.2 Middleware Implementation

```
// RBAC Middleware
export const requirePermission = (permission: Permission) => {
  return async (req: Request, res: Response, next: NextFunction)
=> {
```

Generated by Kimi.ai

```
    const userId = req.user.id;

    // Check cache first
    const cacheKey = `permissions:${userId}`;
    let permissions = await redis.get(cacheKey);

    if (!permissions) {
      // Fetch from DB
      const { rows } = await pgPool.query(
        `SELECT DISTINCT jsonb_array_elements_text(r.permissions)
as perm
         FROM users u
         JOIN user_roles ur ON u.id = ur.user_id
         JOIN roles r ON ur.role_id = r.id
         WHERE u.id = $1 AND u.is_active = true`,
        [userId]
      );

      permissions = rows.map(r => r.perm);
      await redis.setex(cacheKey, 300,
JSON.stringify(permissions));
    } else {
      permissions = JSON.parse(permissions);
    }

    if (!permissions.includes(permission) && !
permissions.includes('*')) {
      return res.status(403).json({ error: 'Insufficient
permissions' });
    }
```

Generated by Kimi.ai

```
    next();
  };
};

// Usage in routes
router.post('/layouts',
  authenticate,
  requirePermission(Permission.LAYOUT_CREATE),
  asyncHandler(layoutController.createLayout)
);
```

---

8. Layout Management System

8.1 JSON Layout Definition Schema

```
// Zod schema for validation
export const LayoutSchema = z.object({
  id: z.string().uuid().optional(),
  title: z.string().min(1).max(255),
  description: z.string().max(1000).optional(),
  organization_id: z.string().uuid(),
  created_by: z.string().uuid(),

  // Grid configuration
  layout_config: z.object({
    breakpoints: z.record(z.number()),
    cols: z.record(z.number()),
    rowHeight: z.number().positive(),
    margin: z.tuple([z.number(), z.number()]),
    containerPadding: z.tuple([z.number(),
z.number()]).optional(),
    compactType: z.enum(['vertical', 'horizontal',
null]).optional()
```

```
  }),

  // Widgets array
  widgets: z.array(z.object({
    id: z.string(),
    type: z.enum(['chart', 'table', 'metric', 'text', 'iframe',
'custom']),
    title: z.string(),
    x: z.number().int().min(0),
    y: z.number().int().min(0),
    w: z.number().int().positive(),
    h: z.number().int().positive(),
    minW: z.number().int().optional(),
    minH: z.number().int().optional(),
    maxW: z.number().int().optional(),
    maxH: z.number().int().optional(),
    static: z.boolean().default(false),
    isDraggable: z.boolean().default(true),
    isResizable: z.boolean().default(true),

    // Widget configuration
    config: z.object({
      dataSource: z.object({
        connector_id: z.string().uuid(),
        query: z.string(),
        parameters: z.array(z.any()).default([]),
        refreshInterval: z.number().min(5000).optional(), //
Minimum 5s
        cacheDuration: z.number().min(0).default(60000),
        timeout: z.number().min(1000).max(300000).default(30000)
      }),
```

Generated by Kimi.ai

```
      visualization: z.record(z.any()).optional(),
      interactions: z.record(z.any()).optional()
    }),

    themeOverrides: z.record(z.record(z.any())).optional()
  })),

  theme: z.object({
    id: z.string(),
    primaryColor: z.string().regex(/^#[0-9A-F]{6}$/i),
    backgroundColor: z.string().regex(/^#[0-9A-F]{6}$/i),
    textColor: z.string().regex(/^#[0-9A-F]{6}$/i)
  }),

  tags: z.array(z.string()).default([]),
  is_public: z.boolean().default(false),
  shared_with: z.array(z.string().uuid()).default([]),
  version: z.number().int().positive().default(1)
});

export type Layout = z.infer<typeof LayoutSchema>;
```

8.2 Layout API Endpoints

```
// Layout Routes with versioning support
router.get('/layouts', authenticate,
asyncHandler(controller.listLayouts));
router.post('/layouts', authenticate,
requirePermission(Permission.LAYOUT_CREATE),
  validate(LayoutSchema), asyncHandler(controller.createLayout));
router.get('/layouts/:id', authenticate,
asyncHandler(controller.getLayout));
router.put('/layouts/:id', authenticate,
```

Generated by Kimi.ai

```
requirePermission(Permission.LAYOUT_UPDATE),
  validate(LayoutSchema), asyncHandler(controller.updateLayout));
router.delete('/layouts/:id', authenticate,
requirePermission(Permission.LAYOUT_DELETE),
  asyncHandler(controller.deleteLayout));
router.post('/layouts/:id/duplicate', authenticate,
asyncHandler(controller.duplicateLayout));
router.get('/layouts/:id/versions', authenticate,
asyncHandler(controller.getVersions));
router.post('/layouts/:id/restore/:version', authenticate,
  requirePermission(Permission.LAYOUT_UPDATE),
asyncHandler(controller.restoreVersion));
router.post('/layouts/:id/export', authenticate,
asyncHandler(controller.exportLayout));
router.post('/layouts/import', authenticate,
requirePermission(Permission.LAYOUT_CREATE),
  asyncHandler(controller.importLayout));
```

---

9. Logging & Monitoring

9.1 Structured Logging (Pino/Winston)

```
// logger.ts
import pino from 'pino';

const logger = pino({
  level: process.env.LOG_LEVEL || 'info',
  formatters: {
    level: (label) => ({ level: label }),
    bindings: (bindings) => ({
      pid: bindings.pid,
      host: bindings.hostname,
      service: process.env.SERVICE_NAME || 'unknown'
```

```
    })
  },
  timestamp: pino.stdTimeFunctions.isoTime,
  redact: {
    paths: ['*.password', '*.token', '*.secret', '*.credentials',
'*.password_hash'],
    remove: true
  },
  transport: process.env.NODE_ENV === 'development'
    ? { target: 'pino-pretty' }
    : { target: 'pino-socket', options: { address: 'logstash',
port: 5000 } }
});

// Request logging middleware
export const requestLogger = (req: Request, res: Response, next:
NextFunction) => {
  const start = Date.now();

  res.on('finish', () => {
    const duration = Date.now() - start;
    logger.info({
      req: {
        id: req.id,
        method: req.method,
        url: req.url,
        headers: {
          'user-agent': req.headers['user-agent'],
          'x-request-id': req.headers['x-request-id']
        }
      },
```

Generated by Kimi.ai

```
      res: {
        statusCode: res.statusCode,
        duration: `${duration}ms`
      },
      user: req.user?.id,
      organization: req.user?.organization_id
    }, 'request completed');
  });

  next();
};

// Audit logging
export const auditLog = async (action: string, entity: string,
details: any) => {
  logger.info({ action, entity, details, type: 'audit' });

  // Also persist to PostgreSQL for compliance
  await pgPool.query(
    `INSERT INTO audit_logs (user_id, organization_id, action,
entity_type,
     entity_id, old_values, new_values, ip_address, user_agent)
     VALUES ($1, $2, $3, $4, $5, $6, $7, $8, $9)`,
    [details.userId, details.orgId, action, entity,
details.entityId,
     details.oldValues, details.newValues, details.ip,
details.userAgent]
  );
};
```

Generated by Kimi.ai

9.2 Health Checks & Metrics

```
// health check endpoint
app.get('/health', async (req, res) => {
  const checks = await Promise.all([
    checkPostgres(),
    checkMongo(),
    checkRedis(),
    checkExternalAPIs()
  ]);

  const isHealthy = checks.every(c => c.status === 'up');

  res.status(isHealthy ? 200 : 503).json({
    status: isHealthy ? 'healthy' : 'unhealthy',
    timestamp: new Date().toISOString(),
    version: process.env.npm_package_version,
    checks: checks.reduce((acc, check) => ({
      ...acc,
      [check.name]: { status: check.status, responseTime:
check.time }
    }), {})
  });
});

// Prometheus metrics endpoint
import promClient from 'prom-client';

const httpRequestDuration = new promClient.Histogram({
  name: 'http_request_duration_seconds',
  help: 'Duration of HTTP requests in seconds',
  labelNames: ['method', 'route', 'status_code'],
```

Generated by Kimi.ai

```
  buckets: [0.1, 0.5, 1, 2, 5]
});

app.get('/metrics', (req, res) => {
  res.set('Content-Type', promClient.register.contentType);
  res.end(promClient.register.metrics());
});
```

---

10. Versioning Strategy

10.1 Semantic Versioning

```
API Versioning: /api/v1/layouts
Service Versions: layout-service:v1.2.3
Database Migrations: Sequential with rollback support
```

10.2 Database Migration Strategy

```
// Using node-pg-migrate or TypeORM
import { MigrationInterface, QueryRunner } from "typeorm";

export class AddWidgetRefreshPolicy1710000000000 implements
MigrationInterface {
    name = 'AddWidgetRefreshPolicy1710000000000';

    public async up(queryRunner: QueryRunner): Promise<void> {
        await queryRunner.query(`
            ALTER TABLE widgets
            ADD COLUMN refresh_policy JSONB DEFAULT '{"type":
"interval", "value": 300000}';
        `);

        // Backfill existing data
        await queryRunner.query(`
            UPDATE widgets
```

```
            SET refresh_policy = jsonb_build_object('type',
'interval', 'value', refresh_interval)
            WHERE refresh_interval IS NOT NULL;
        `);
    }

    public async down(queryRunner: QueryRunner): Promise<void> {
        await queryRunner.query(`ALTER TABLE widgets DROP COLUMN
refresh_policy;`);
    }
}
```

10.3 Layout Versioning

Every layout update creates a version snapshot:

```
async function createLayoutVersion(layoutId: string, userId:
string, summary: string) {
  const layout = await getLayout(layoutId);
  const versionNumber = layout.version + 1;

  // Store snapshot in PostgreSQL
  await pgPool.query(
    `INSERT INTO layout_versions (layout_id, version_number,
created_by,
     change_summary, layout_snapshot)
     VALUES ($1, $2, $3, $4, $5)`,
    [layoutId, versionNumber, userId, summary,
JSON.stringify(layout)]
  );

  // Update current version in MongoDB
  await mongoClient.db().collection('layouts').updateOne(
    { _id: new ObjectId(layoutId) },
```

```
    { $set: { version: versionNumber, updated_at: new Date() } }
  );

  // Keep only last 50 versions (cleanup in background)
  cleanupOldVersions(layoutId);

  return versionNumber;
}
```

---

11. Security Considerations

Layer	Measures
Transport	TLS 1.3, HSTS, Secure cookies
Authentication	JWT with RS256, refresh tokens, MFA support
Authorization	RBAC with principle of least privilege
Data	AES-256 encryption for credentials, field-level encryption for PII
API	Rate limiting (100 req/min anonymous, 1000 req/min authenticated), input validation
Containers	Non-root users, read-only filesystems, security scanning with Trivy
Secrets	Docker Secrets / Kubernetes Secrets, never commit to repo
CORS	Whitelist origins, strict policy

---

12. Deployment Architecture

```
Production (Kubernetes)
├── Namespace: dashboard-prod
│   ├── Deployments: 3 replicas per service
```

Generated by Kimi.ai

```
│   ├── HPA: Auto-scale 3-10 pods based on CPU/memory
│   ├── Ingress: TLS termination, path-based routing
│   ├── Network Policies: Service-to-service rules
│   └── Pod Security Policies: Non-root, read-only root fs
├── Namespace: dashboard-monitoring
│   ├── Prometheus: Metrics collection
│   ├── Grafana: Visualization
│   ├── ELK Stack: Centralized logging
│   └── Jaeger: Distributed tracing
└── CI/CD: GitHub Actions → Build → Test → Scan → Deploy
```

---

13. Development Workflow

```bash
# Clone and start development environment
git clone https://github.com/org/dashboard-platform.git
cd dashboard-platform

# Start all services
docker-compose up -d

# Run migrations
docker-compose exec auth-service npm run migrate

# Seed data
docker-compose exec auth-service npm run seed

# View logs
docker-compose logs -f [service-name]

# Run tests
docker-compose exec auth-service npm test
docker-compose exec frontend npm test
```

Generated by Kimi.ai

14. API Documentation

OpenAPI 3.0 specification available at `/api/docs` with Swagger UI integration. All endpoints documented with request/response schemas and authentication requirements.

15. Conclusion

This Modular Dashboard Platform provides:

✅ **True Modularity**: Add widgets on-the-fly with drag-drop-resize

✅ **Async-First**: All I/O operations are non-blocking

✅ **Multi-Database**: Optimized storage per data type (Postgres, MongoDB, Redis)

✅ **Dockerized**: Multi-stage builds, health checks, security hardening

✅ **Themable**: CSS variables + runtime theme switching

✅ **RBAC**: Granular permissions with role inheritance

✅ **Versioned**: Layout snapshots + database migrations

✅ **Observable**: Structured logging, metrics, distributed tracing

Next Steps: 1. Implement core services (Auth + Layout) 2. Create React-Grid-Layout integration 3. Build connector adapters for PostgreSQL, MySQL, REST APIs 4. Implement WebSocket real-time updates 5. Add comprehensive test suite (unit, integration, E2E)

Document Version: 1.0
Last Updated: 2026-03-13
Author: Technical Architecture Team

Generated by Kimi.ai